

Efficient Data Storage Systems

University of Minho

Guide 8

Benchmarking

2026

The main goal of this guide is to set up a benchmark framework, understand its workloads, and use them to assess the performance of different file system implementations. Namely, we will use the Flexible I/O Tester (*fio*) benchmark tool to characterize the performance of different FUSE-based implementations used in the classes.

1 Setup

Use the VM from previous guides to set up the benchmark environment.

1. Install required packages.

```
$ sudo apt install fio
```

2. Check the *fio* documentation at:

- Running *fio*: https://fio.readthedocs.io/en/latest/fio_doc.html#running-fio
- How *fio* works: https://fio.readthedocs.io/en/latest/fio_doc.html#how-fio-works

Hint: For the following exercises, look at the pages on *I/O type*, *block size*, *I/O size*, *I/O engine*, and *target file/device*.

2 Exercises

Let's start by measuring the performance of the VM file system and disk. Run the benchmark on a folder belonging to the VM's base file system (e.g., `/home/vagrant/`).

Important: Save the outputs of the experiments as these will be important later on!

1. Try *fio* with a sequential write I/O pattern (flag `-rw=write`), using a 4 KB block size (flag `-bs=4k`) and a total data size of 1 GB (flag `-size=1g`). Use the flag `--ioengine=psync` to force the use of *pread*, *pwrite* operations. Use the flag `--name` to specify the test name.

```
$ fio --rw=write --bs=4k --size=1g --ioengine=psync --name=seqwr-bs4k-s1g
```

Refer to https://fio.readthedocs.io/en/latest/fio_doc.html#interpreting-the-output for guidance on interpreting *fio*'s output.

2. Run the same experiment but with a random I/O access pattern instead. Do you notice any difference in performance?

Hint: Use the flag `--rw` (or `--readwrite`) to vary the I/O pattern (sequential vs. random) and the type of I/O operation (read, write, or mixed read/write).

3. Now run 2 additional experiments for sequential and random read requests, respectively. Again, do you notice any difference in performance?

Pick up two FUSE-based implementations developed in the previous guides. For instance, you can use the FUSE pass-through implementation (Guide 2) and the remote implementation (Guide 6 or 7).

Note: If your remote implementation assumes fixed-size reads of 4KB, then add the following code line to your `xmp_init` function to disable readahead in FUSE.

```
conn->max_readahead = 0;
```

Repeat the above experiments for each FUSE-based implementation, and answer the following questions:

1. How much performance overhead does FUSE introduce for different read and write workloads?
2. How much performance overhead does a remote FUSE-based implementation introduce for different workloads?

3 Extras / Homework

1. Try out changing other configurations of `fio` experiments. Here are some **Hints**:
 - Use the flag `--bs` to set the block size (e.g., 1 KB, 8 KB) and the flag `--size` to specify the total data size (e.g., 1MB, 1 GB, 10 GB).
 - Use the flag `--numjobs` to run multiple jobs concurrently and observe the effect of parallelism.
 - Use the flag `--directory` to select the directory where `fio` writes its data.
 - Use the flag `--ioengine` to explore different I/O engines. For example, the `sync` engine uses read/write syscalls with `lseek` for positioning, while the `psync` engine uses `pread/pwrite` syscalls for direct positioning.
2. Complement `fio`'s results with other tools such as `top` or `dstat` to monitor other system resources.
3. Take a look at other benchmarks, such as *Filebench*.

Learning outcomes

Experiment the `fio` benchmarking framework. Understand how to evaluate storage systems with synthetic benchmarks. Experiment with file system evaluation.